# WuWei データ設計書

# Table of Contents


1. 目的 . . . 1
2. 基本方針 . . . 2
3. 識別子 . . . 3
4. 共通型 . . . 4
    4.1. Audit . . . 4
    4.2. RichText . . . 4
    4.3. Transform . . . 4
    4.4. Style . . . 4
5. Note . . . 6
    5.1. Note ライフサイクル . . . 6
6. Page . . . 8
    6.1. Page 番号 . . . 8
    6.2. Page 追加 . . . 9
    6.3. Page 削除 . . . 9
    6.4. Page 複製 . . . 9
7. Resource . . . 10
    7.1. Resource 定義 . . . 10
    7.2. media.kind . . . 11
    7.3. Resource URI の解決順序 . . . 11
    7.4. iframe 表示 . . . 11
    7.5. storage . . . 12
    7.6. Resource の同一判定 . . . 13
    7.7. 空 URI による上書き禁止 . . . 13
8. Content と ResourceView . . . 14
9. Node . . . 15
    9.1. 共通 Node . . . 15
    9.2. Topic . . . 15
    9.3. Content . . . 15
    9.4. Memo . . . 15
10. Link . . . 16
11. Group . . . 17
    11.1. simple Group . . . 17
    11.2. horizontal / vertical Group . . . 17
    11.3. timeline Group . . . 17
12. 保存ファイル構造 . . . 18
    12.1. Note . . . 18
    12.2. Resource library . . . 18
    12.3. Upload . . . 18

13. save-note / load-note . . . 20
13.1. save-note . . . 20
13.2. load-note . . . 20
14. wuwei.home . . . 21
15. 検索対象 . . . 22
16. 禁止する旧モデル属性 . . . 23
17. 公開 Note と閲覧専用モード . . . 25
18. Page thumbnail の運用 . . . 26
19. Home 画面の Resource 表示 . . . 27
20. ヘッダーメニュー操作とデータ更新 . . . 28
20.1. Note メニュー . . . 28
20.2. Page メニュー . . . 28
20.3. New メニュー . . . 28
20.4. Group メニュー . . . 29
21. 2026-04-29 補足: ノート入出力 API とメニューコマンド . . . 30

# Chapter 1. 目的

この文書は、WuWei app2 で扱う Note、Page、Node、Link、Group、Resource のデータモデル、保存時のファイル配置、入出力 API、メニュー操作との対応を一体として定義する。

`manual/data` 配下のデータ関連仕様は、この `data.adoc` を正本とする。 旧来の `data_model.adoc` と `data_design.adoc` に分かれていた内容は、本書に統合する。

旧バージョンの Note データは `wuwei.note` が読み込む時点で新モデルへ変換する。 変換後の実行時データおよび保存データには、旧モデル由来の属性を残さない。 旧モデル属性が残っている場合は互換処理を行わず、エラーとして扱う。

# Chapter 2. 基本方針

- Note は `resources[]` と `pages[]` を持つ。
- Page は `nodes`、`links`、`groups` を持つ。
- Content ノードは Resource 本体を直接保持せず、`resourceRef` で `resources[].id` を参照する。
- 同じ Resource は複数の Content ノードから参照できる。
- 同一 Resource の参照ページ、シート、時刻範囲、fragment などの違いは Content 側の `resourceView` に保持する。
- 外部 Web Resource は原則として実体コピーせず、メタデータ、URI、埋め込み情報を保持する。
- 自己アップロード Resource は Resource ライブラリに登録し、必要に応じて Note snapshot にコピーする。
- Page ID は `_page_uuid` とし、ページ番号は `pages[]` の表示順から計算する。
- Link の矢印は `routing.startArrow` と `routing.endArrow` で表す。
- Group 種別は `simple`、`horizontal`、`vertical`、`timeline` とする。
- Group の表示/非表示は、Group 全体を一括で制御する。
- 新規 Note は作成時点で `note_id` を確定する。
- Note の「新規」と「破棄」は別操作として扱う。

# Chapter 3. 識別子

ID は UUID 文字列を基本とし、WuWei 内では先頭に _ を付ける。

```
_xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
```

主な ID は次のとおりである。

| ID | 用途 |
|---|---|
| `note_id` | Note を識別する。 |
| `page.id` | Page を識別する恒久 ID。ページ番号ではない。 |
| `node.id` | Node を識別する。 |
| `link.id` | Link を識別する。 |
| `group.id` | Group を識別する。 |
| `resource.id` | Resource を識別する。 |

# Chapter 4. 共通型

## 4.1. Audit

```
{
  "owner": "sambuichi",
  "createdBy": "_user_uuid",
  "createdAt": "2026-04-27T12:00:00+09:00",
  "lastModifiedBy": "",
  "lastModifiedAt": ""
}
```

## 4.2. RichText

説明、注釈、Memo 本文などの文字列は、形式と本文を分けて保持する。

```
{
  "format": "asciidoc",
  "body": "== TITLE"
}
```

`format` は `plain/text`、`asciidoc`、`markdown`、`html` などを指定できる。 表示時は `format` に従ってレンダリングする。 検索時は markup 制御文字を除いた本文相当の文字列を対象にする。

## 4.3. Transform

Page または描画キャンバスの表示位置と倍率を表す。

```
{
  "x": 0,
  "y": 0,
  "scale": 1
}
```

## 4.4. Style

```
{
  "fill": "#FEFEFE",
  "font": {
    "family": "sans-serif",
    "size": 14,
    "color": "#000000",
    "align": "center"
```

```
  },
  "line": {
    "kind": "SOLID",
    "color": "#666666",
    "width": 1
  }
}
```

編集画面の HTML id や変数名では、階層名を _ で連結した形式を用いる。 例: `style.fill` は `style_fill`、`style.font.size` は `style_font_size`。

# Chapter 5. Note

Note はページ、参照 Resource 定義、監査情報を持つ。 Note 単体をファイルとして受け渡しても再現できるように、参照 Resource の定義コピーを `resources[]` に保持する。

```
{
  "note_id": "_note_uuid",
  "note_name": "Note label",
  "description": "Note description",
  "currentPage": "_page_uuid",
  "resources": [],
  "pages": [],
  "audit": {}
}
```

| 項目 | 説明 |
|---|---|
| `note_id` | Note ID。 |
| `note_name` | Note 名。 |
| `description` | Note 全体の説明。 |
| `currentPage` | 現在表示している Page ID。ページ番号ではなく `_page_uuid` を保存する。 |
| `resources` | この Note が参照する Resource 定義のコピー。 |
| `pages` | 表示順に並ぶ Page 配列。 |
| `audit` | 監査情報。 |

## 5.1. Note ライフサイクル

Note は、次のいずれかの状態から編集を開始する。

| 開始方法 | 仕様 |
|---|---|
| 新規 Note | `wuwei.note.newNote()` が空の Note を作成し、その時点で新しい `note_id` を割り当てる。編集中だった Note に対して保存、削除、破棄などの副作用は行わない。 |
| 保存済み Note の読み込み | `load-note` で保存済み `note.json` を読み込み、保存されている `note_id` を引き継ぐ。 |
| portable bundle のインポート | `import-note` が bundle を復元し、Note と Resource snapshot を保存する。復元後の保存先ユーザーは現在ログイン中のユーザーとする。 |

`note_id` は Note 作成時または読み込み時に必ず確定しているものとして扱う。 これにより、アップロード Resource の Note snapshot 保存先を決定できる。

Note メニューの「新規」は、現在編集中の Note を保存も削除もせず、新しい空 Note に切り替える操作である。 編集中 Note を明示的に捨てる操作は「破棄」として別に定義する。 「破棄」は確認後に現在の編集状態を破棄し、新しい空 Note を作成する。 保存済み Note のサーバー上の実体や Resource ライブ

ラリは削除しない。

# Chapter 6. Page

Page は、1 枚の作業面を表す。 Page は Node、Link、Group、表示位置、thumbnail、監査情報を持つ。

Page にはページ番号を恒久 ID として持たせない。 Page の恒久 ID は `id` に保持する `_page_uuid` である。 ページ番号は Note の `pages[]` 内の順序から 1 始まりで計算される表示用番号である。 ページの追加、削除、並べ替えにより、同じ Page ID でも表示用ページ番号は変わり得る。

```
{
  "id": "_page_uuid",
  "label": "Page label",
  "description": {
    "format": "plain/text",
    "body": ""
  },
  "nodes": [],
  "links": [],
  "groups": [],
  "transform": {
    "x": 0,
    "y": 0,
    "scale": 1
  },
  "thumbnail": null,
  "audit": {}
}
```

| 項目 | 説明 |
|---|---|
| `id` | Page の恒久 ID。 |
| `label` | Page 名。画面表示や Page 一覧で使用する。 |
| `description` | Page の説明。 |
| `nodes` | Page 上に配置される Node の配列。 |
| `links` | Page 上の Node 間を結ぶ Link の配列。 |
| `groups` | Page 上の Group 定義の配列。 |
| `transform` | Page の表示位置と倍率。 |
| `thumbnail` | Page 一覧用の縮小表示。 |
| `audit` | 監査情報。 |

## 6.1. Page 番号

ページ番号は保存値ではなく、`pages[]` の現在の順序から求める。

```
pageNumber = pages.indexOf(page) + 1
```

Page 一覧、draw 上の表示、ページ操作 UI ではこの表示番号を使う。 保存データの参照や `currentPage` には Page ID を使う。

## 6.2. Page 追加

Page メニューの「新規ページ」は、新しい Page ID を採番し、`pages[]` の末尾に空の Page を追加する。 追加直後は、新しく作成された Page を `currentPage` に設定する。

```
{
  "id": "_new_page_uuid",
  "label": "",
  "description": {
    "format": "plain/text",
    "body": ""
  },
  "nodes": [],
  "links": [],
  "groups": [],
  "transform": {
    "x": 0,
    "y": 0,
    "scale": 1
  },
  "thumbnail": null,
  "audit": {}
}
```

ページ追加時に既存 Page の番号を ID として再利用してはならない。 削除された Page の表示番号が空いていても、Page ID は常に新しい UUID とする。

## 6.3. Page 削除

Page 削除では、対象 Page を `pages[]` から取り除く。 削除後のページ番号は残った `pages[]` の順序から再計算する。 削除対象が `currentPage` の場合は、残っている先頭 Page を新しい `currentPage` とする。 すべての Page が削除される場合は、空の Page を 1 件作成する。

## 6.4. Page 複製

Page 複製では、Page ID、Node ID、Link ID、Group ID を新しく採番する。 複製元 Page の Node/Link/Group をそのまま ID 付きで共有してはならない。 Content が参照する Resource は同じ Resource ID を共有してよい。

# Chapter 7. Resource

Resource は Content ノードから参照される再利用可能なコンテンツ定義である。 Resource は大きく、外部 Web Resource と Uploaded Resource に分ける。

| 分類 | 説明 |
|---|---|
| 外部 Web Resource | Web page、blog、YouTube、Vimeo、Entity site、公開 PDF URL など、他者の公開情報を参照する。 |
| Uploaded Resource | PDF、Office、text、video、audio、photo など、ユーザーがアップロードした自己登録情報。 |

## 7.1. Resource 定義

```
{
  "id": "_resource_uuid",
  "type": "Resource",
  "origin": {
    "type": "userRegistered",
    "subtype": "",
    "provider": ""
  },
  "identity": {
    "title": "Document title",
    "uri": "",
    "canonicalUri": ""
  },
  "media": {
    "kind": "document",
    "mimeType": "application/pdf",
    "downloadable": true,
    "duration": null
  },
  "viewer": {
    "supportedModes": ["infoPane", "newTab", "newWindow", "download"],
    "defaultMode": "infoPane",
    "embed": {
      "uri": "",
      "html": ""
    }
  },
  "storage": {
    "managed": true,
    "copyPolicy": "snapshot",
    "sourcePath": "yyyy/mm/dd/{uploaded_file_uuid}/original.pdf",
    "primaryPath": "data/{user_uuid}/resource/yyyy/mm/dd/{resource_uuid}",
    "snapshotPath": "",
    "files": []
  },
```

```
  "snapshotSources": {},
  "rights": {
    "owner": "",
    "copyright": "",
    "license": "",
    "attribution": ""
  },
  "audit": {}
}
```

## 7.2. media.kind

| 値 | 説明 |
|---|---|
| `webpage` | 通常の Web ページ。 |
| `video` | YouTube、Vimeo、アップロード動画。 |
| `audio` | 音声。 |
| `document` | PDF、Office、text などの文書。 |
| `image` | 画像。 |
| `general` | 特定の viewer を持たない一般 Resource。 |

YouTube と Vimeo は `webpage` ではなく `video` として扱う。

## 7.3. Resource URI の解決順序

info ペイン、外部ウィンドウ表示、Resource 追加処理で表示 URI を求めるときは、次の順序で解決する。

```
resource.identity.uri
resource.viewer.embed.uri
resource.snapshotSources.previewUri
resource.identity.canonicalUri
resource.snapshotSources.originalUri
```

`identity.uri` は通常表示 URL である。 YouTube や Vimeo など、通常 URL と iframe 埋め込み URL が異なる場合は、通常 URL を `identity.uri`、iframe 用 URL を `viewer.embed.uri` に保持する。

URI が完全に空の Resource は、info ペインに表示対象を持たない。 表示側は URL を推測してはならない。

## 7.4. iframe 表示

info ペインで Web page、YouTube、Vimeo などを表示する場合、PHP proxy は使用しない。 `server/proxy.php` などの PHP スクリプトを iframe の src に設定してはならない。 サーバー CGI は shell script を基本とし、PHP 実行環境を前提にしない。

Web page は URI を直接 iframe に設定する。 対象サイトが `X-Frame-Options` や CSP により iframe 表示を拒否する場合は、iframe を隠し、URL と拒否メッセージを表示する。 この場合でも、新しいタブまたは別ウィンドウで開くリンクを提供する。

## 7.5. storage

`storage` は Resource の保存場所とコピー方針を定義する。

| 項目 | 説明 |
|---|---|
| `managed` | WuWei がローカルファイルを管理する場合は `true`。 |
| `copyPolicy` | `snapshot` または `metadataOnly`。 |
| `sourcePath` | upload 領域からの相対パス。例: `2026/04/26/{uploaded_file_uuid}/document.pdf`。 |
| `primaryPath` | Resource ライブラリ側の保存ディレクトリ。 |
| `snapshotPath` | Note snapshot 側の Resource 保存ディレクトリ。 |
| `files` | Resource に関連するファイル一覧。 |

`storage.files[]` の例:

```
[
  {
    "role": "original",
    "path": "yyyy/mm/dd/{uploaded_file_uuid}/original.pdf",
    "sourcePath":
"C:/Apache24/htdocs/wu_wei2/data/{user_uuid}/upload/yyyy/mm/dd/{uploaded_file_uuid}/or
iginal.pdf",
    "mimeType": "application/pdf",
    "size": 123456,
    "sha256": "..."
  },
  {
    "role": "thumbnail",
    "path": "thumbnail.png",
    "mimeType": "image/png",
    "size": 12345,
    "sha256": "..."
  },
  {
    "role": "preview",
    "path": "preview.pdf",
    "mimeType": "application/pdf",
    "size": 123456,
    "sha256": "..."
  }
]
```

## 7.6. Resource の同一判定

Resource ライブラリでは、同じ実体を重複保存しない。 同一判定は次の優先順位で行う。

```
resource.identity.canonicalUri
resource.identity.uri
resource.storage.sourcePath
```

upload ファイルでは、`yyyy/mm/dd/{uploaded_file_uuid}/{filename}` に実体を保存する。 同一日付内で同じ `{filename}` が再アップロードされた場合は、既存の `{uploaded_file_uuid}` 配下を更新し、sha256 が異なっても同じファイルの更新とみなす。 Note snapshot 配下では、同じ Resource であっても Note ごとにコピーを持ってよい。 Note 単体を受け渡しても再現可能にするためである。

## 7.7. 空 URI による上書き禁止

既存 Resource に URI が定義されている場合、Note 保存または Resource 更新で渡された Resource 定義の URI が空であっても、既存 URI を空で上書きしてはならない。

保護対象は次の項目である。

```
resource.identity.uri
resource.identity.canonicalUri
resource.viewer.embed.uri
resource.snapshotSources.previewUri
resource.snapshotSources.originalUri
resource.snapshotSources.thumbnailUri
```

# Chapter 8. Content と ResourceView

Content は Resource を参照する Node である。 Resource の実体や定義を Content ノードに展開して永続保存しない。

```
{
  "id": "_content_uuid",
  "type": "Content",
  "resourceRef": "_resource_uuid",
  "resourceView": {
    "mode": "documentPage",
    "page": 12,
    "fragment": "#page=12",
    "time": {
      "start": "",
      "end": ""
    }
  },
  "x": 0,
  "y": 0,
  "shape": "RECTANGLE",
  "size": {
    "width": 120,
    "height": 100
  },
  "visible": true,
  "label": "Content label",
  "description": {
    "format": "asciidoc",
    "body": ""
  },
  "style": {},
  "audit": {}
}
```

`label` と `description` は Note 上の解釈や注釈であり、Resource の `identity.title` とは独立している。

# Chapter 9. Node

## 9.1. 共通 Node

```
{
  "id": "_node_uuid",
  "type": "Topic",
  "x": 0,
  "y": 0,
  "shape": "RECTANGLE",
  "size": {
    "width": 120,
    "height": 40
  },
  "visible": true,
  "label": "",
  "description": {
    "format": "asciidoc",
    "body": ""
  },
  "style": {},
  "audit": {}
}
```

## 9.2. Topic

Topic は概念、タグ、話題を表す Node である。

## 9.3. Content

Content は Resource を参照する Node である。 `resourceRef` は必須である。

## 9.4. Memo

Memo は付箋紙として表示する Node である。 本文は `description.body` に保存し、`description.format` に従ってレンダリングする。

# Chapter 10. Link

Link は Node 間の接続を表す。

```
{
  "id": "_link_uuid",
  "type": "Link",
  "from": "_source_node_uuid",
  "to": "_target_node_uuid",
  "shape": "NORMAL",
  "x": 0,
  "y": 0,
  "x2": 0,
  "y2": 0,
  "visible": true,
  "label": "",
  "description": {
    "format": "plain/text",
    "body": ""
  },
  "style": {},
  "routing": {
    "startArrow": {
      "kind": "NONE",
      "size": 12
    },
    "endArrow": {
      "kind": "ARROW",
      "size": 12
    }
  },
  "audit": {}
}
```

`shape` は `NORMAL`、`HORIZONTAL`、`HORIZONTAL2`、`VERTICAL`、`VERTICAL2` などを使用できる。 `x` と `y` は第 1 制御点である。 `HORIZONTAL2` と `VERTICAL2` は第 2 制御点として `x2` と `y2` を保存する。 `NORMAL`、`HORIZONTAL`、`VERTICAL` では通常 `x2` と `y2` は使用しない。

# Chapter 11. Group

Group は複数の Node、Link、または Group 軸をまとめて扱うための定義である。

```
{
  "id": "_group_uuid",
  "type": "simple",
  "visible": true,
  "members": [
    {
      "nodeId": "_node_uuid",
      "role": "member",
      "order": 1
    }
  ],
  "spine": {},
  "style": {},
  "audit": {}
}
```

Group 種別は次のいずれかとする。

- `simple`
- `horizontal`
- `vertical`
- `timeline`

Group の表示/非表示は Group 全体に対して適用する。 Group のメンバー Node のいずれかを Group 操作として非表示にする場合、その Group 全体を非表示にする。 同じ規則は timeline Group にも適用する。

## 11.1. simple Group

simple Group はメンバー全体を囲むグループである。 囲み線はメンバーの現在位置に追従して描画する。

## 11.2. horizontal / vertical Group

horizontal Group と vertical Group は軸を持つグループである。 軸は表示対象であり、ドラッグ操作により移動できる。 simulation モードでは、Group の形状を不用意に変形しない。

## 11.3. timeline Group

timeline Group は時間軸を持つグループである。 開始時刻、終了時刻、軸方向、表示位置、対象 media への参照を保持する。 timeline Group も Group 一括表示/非表示の対象である。

# Chapter 12. 保存ファイル構造

## 12.1. Note

```
data/{user_uuid}/note/yyyy/mm/dd/{note_uuid}/note.json
data/{user_uuid}/note/yyyy/mm/dd/{note_uuid}/resource/{resource_uuid}/resource.json
data/{user_uuid}/note/yyyy/mm/dd/{note_uuid}/resource/{resource_uuid}/original.*
data/{user_uuid}/note/yyyy/mm/dd/{note_uuid}/resource/{resource_uuid}/thumbnail.*
data/{user_uuid}/note/yyyy/mm/dd/{note_uuid}/resource/{resource_uuid}/preview.pdf
```

`note.json` は metadata と Note JSON 本体を保持する。 Note JSON 本体は `json_base64` として保存できる。

```
format_version 2
id _note_uuid
user_id _user_uuid
name Note label
description Note description
thumbnail ...
saved_at 2026-04-27T12:00:00+0900
json_encoding base64
json_base64 eyJub3RlX2lkIjoiX25vdGVfdXVpZCJ9...
```

`json_base64` を decode した Note 本体は、この文書の Note 定義に従う。

## 12.2. Resource library

```
data/{user_uuid}/resource/yyyy/mm/dd/{resource_uuid}/resource.json
data/{user_uuid}/resource/yyyy/mm/dd/{resource_uuid}/original.*
data/{user_uuid}/resource/yyyy/mm/dd/{resource_uuid}/thumbnail.*
data/{user_uuid}/resource/yyyy/mm/dd/{resource_uuid}/preview.pdf
```

同じ Resource は Resource library では重複保存しない。 一方、Note snapshot 配下では Note ごとにコピーを持ってよい。

## 12.3. Upload

upload 済みファイルの実体は次の場所にある。

```
data/{user_uuid}/upload/yyyy/mm/dd/{uploaded_file_uuid}/{filename}
```

Resource は upload 実体を直接複製するのではなく、`storage.sourcePath` や `storage.files[].sourcePath` で参照する。 Note snapshot へコピーするときは、Resource ごとのディレ

クトリ直下に `original.`、`thumbnail.`、`preview.pdf` としてロール名ベースで保存する。

# Chapter 13. save-note / load-note

## 13.1. save-note

`save-note` は Note 保存時に、Note 内で新規作成または更新された Resource も保存する。 既存 Resource を参照するだけの場合でも、Note 側には Resource 定義コピーを `resources[]` として保持する。

自己アップロードファイルは Resource library と Note snapshot の両方に必要なファイルを持てる。 外部 Web URL は原則コピーせず、metadata と embed 情報のみを保持する。

## 13.2. load-note

`load-note` は Note ファイルを解決して読み込む。 Note snapshot 配下の `resource.json` は Note 一覧の対象に含めない。

読込後、`wuwei.note` は `resources[]` を `id` で索引化し、各 Content ノードの `resourceRef` に対応する Resource を実行時用 `node.resource` として復元する。 復元された `node.resource` は実行時専用であり、保存時の Note JSON には残さない。

# Chapter 14. wuwei.home

`wuwei.home` は Resource library に登録された Resource の一覧を表示し、選択した Resource を Note に追加するための画面である。 対象は `data/{user_uuid}/resource/yyyy/mm/dd/{resource_uuid}/resource.json` とする。 upload ディレクトリやテストデータを直接一覧対象にしない。

ギャラリーと選択中コンテンツ preview は次の順序で表示 URI を解決する。

```
resource.viewer.embed.uri
resource.snapshotSources.previewUri
resource.identity.uri
resource.identity.canonicalUri
storage.files[].role = "preview"
storage.files[].role = "thumbnail"
```

`Add to note` は、選択中 Resource を編集中 Note に Content ノードとして追加する。 `新規ノート` は、新しい Note を作成してから選択中 Resource を Content ノードとして追加する。

ギャラリーのカードには Resource の thumbnail または preview を表示する。 Note の一覧やギャラリーで Note を表示する場合は、Note の `currentPage` に対応する Page を優先して thumbnail 表示する。

# Chapter 15. 検索対象

Resource 検索の初期対象は次の項目とする。

- `resource.label`
- `resource.description.body`
- `resource.identity.title`
- `resource.storage.sourcePath`
- `resource.storage.files[].path`
- `resource.identity.uri`
- `resource.identity.canonicalUri`

`media.kind`、`media.mimeType`、`rights.owner`、`rights.copyright`、`rights.license`、`rights.attribution` は将来の検索対象候補とし、初期実装では検索対象外とする。

# Chapter 16. 禁止する旧モデル属性

`wuwei.note` の読み込み変換後、次の属性は実行時データおよび保存データに残してはならない。

- `node.name`
- `node.resource`
- `node.media`
- `node.uri`
- `node.url`
- `node.format`
- `node.hidden`
- `node.view`
- `link.source`
- `link.target`
- `link.hidden`
- `link.view`
- `note.resources` がオブジェクト map である構造
- `page.pp`
- ページ番号を Page ID として扱う構造
- `associations`
- `idx`
- `owner_id`
- `ownerId`
- `userId`
- `userID`
- `nFont_size`
- `nFont_color`
- `background`

対応する新モデルの保存先は次のとおりである。

| 旧属性 | 新モデル |
|---|---|
| `node.name` | `node.label` |
| `node.media.kind` | `resource.media.kind` |
| `node.media.mimeType` | `resource.media.mimeType` |
| `node.url` / `node.uri` | `resource.identity.uri`、`resource.identity.canonicalUri` |
| `node.format` | `node.description.format` |

| 旧属性 | 新モデル |
| --- | --- |
| `node.content` | `node.description.body` |
| `nFont_size` | `node.style.font.size` |
| `nFont_color` | `node.style.font.color` |
| `background` | `node.style.fill` |

# Chapter 17. 公開 Note と閲覧専用モード

Publish は、編集中の Note を公開 URL から閲覧できるようにする操作である。 Publish 成功後は、画面にモーダルメッセージを表示し、利用者が次の操作を選択できるようにする。

- 公開完了を確認する。
- 公開 URL を確認する。
- 公開 URL をコピーする。
- 利用者が明示的に選択した場合だけ、公開 URL を新規タブで開く。

新規タブを開く処理は、利用者が「新規タブで開く」を押したときだけ `window.open(publicUrl, '_blank', 'noopener')` を実行する。 Publish 成功だけで自動的に新規タブを開いてはならない。

公開 URL で読み込まれた Note は、URL パラメータまたは読み込みモードにより `published/viewOnly` と判定する。 実行時状態は次のように扱う。

| 状態 | 意味 |
|---|---|
| `state.published` | 公開 URL から読み込まれた Note であることを示す。 |
| `state.viewOnly` | 編集不可、保存不可、ダウンロード不可の閲覧専用であることを示す。 |
| `graph.mode = "view"` | 描画グラフを閲覧モードとして操作する。 |

閲覧専用モードでは、次の操作を無効化する。

- Node、Link、Group、Timeline、Page の作成、編集、削除。
- Note の保存。
- Note の Publish。
- Note または Resource のダウンロード。
- 編集系コンテキストメニュー。
- ヘッダーメニューの編集系アイコン。

`published` および `viewOnly` は実行時の表示・操作制御であり、Note JSON の恒久属性として保存しない。 公開された Note のデータ構造は通常の Note と同一である。

# Chapter 18. Page thumbnail の運用

Page は、一覧表示、ページ番号ホバー、Note 選択表示のために `page.thumbnail` を持つ。
`page.thumbnail` は、右上の縮小図と同じ内容を表す 200 x 200 の SVG として保存する。

thumbnail の更新タイミングは次のとおりとする。

- Page を切り替える直前。
- Page を追加する直前または直後。
- Page をコピーする直前または直後。
- Note を保存する直前。
- Page 一覧を開く前。

Page 一覧画面と左下のページ番号ホバー時のサムネイル表示は、画面上の SVG をその場で縮小表示するのではなく、保存済みの `page.thumbnail` を利用する。 これにより、Page 切り替え後や Page 追加後でも、一覧表示の形状と背景が安定する。

Note の代表 thumbnail が必要な場合は、`currentPage` が指す Page の `thumbnail` を利用する。 Note 直下に Page と重複する thumbnail を保存することは必須ではない。

# Chapter 19. Home 画面の Resource 表示

`wuwei.home` は、`data/{user_uuid}/resource/yyyy/mm/dd/{resource_uuid}/` に登録された Resource を対象とする。 テスト用の固定データではなく、Resource library の `resource.json` と関連ファイルを読み込んで表示する。

ギャラリーには、Resource の種別に応じて次の内容を表示する。

| **Resource** 種別 | 表示内容 |
|---|---|
| PDF、Office、画像などのファイル | `storage.files[]` の `thumbnail` または `preview` を優先して表示する。 |
| Web page | `identity.uri` または `identity.canonicalUri` を iframe で表示する。 |
| YouTube、Vimeo | 動画用 iframe として表示する。ホスト名から YouTube または Vimeo と判定できる場合、`webpage` ではなく `video` として扱う。 |
| iframe 表示を拒否する外部サイト | 拒否されたことを示すメッセージと URL を表示する。 |

選択中のコンテンツ欄には、ギャラリーで選択した Resource の preview、label、description、credit、source URL を表示する。 description は `description.format` に応じて、asciidoc、markdown、html、plain text の表示を切り替える。

Home 画面の検索対象は、初期仕様では次の属性とする。

- `resource.label`
- `resource.description.body`
- `resource.identity.title`
- `resource.storage.sourcePath`
- `resource.storage.files[].path`
- `resource.identity.uri`
- `resource.identity.canonicalUri`

`media.kind`、`media.mimeType`、`rights.owner`、`rights.copyright`、`rights.license`、`rights.attribution` は、今後の検索対象候補とする。

# Chapter 20. ヘッダーメニュー操作とデータ更新

ヘッダーメニューは、現在の Note、Page、選択状態、閲覧モードに応じて操作可能な項目を切り替える。

## 20.1. Note メニュー

| 操作 | データ更新 |
|---|---|
| 新規 | 新しい Note を作成する。現在編集中の Note には保存、削除、破棄などの処理を行わない。 |
| 開く | 保存済み Note 一覧から Note を読み込む。 |
| 保存 | Note JSON、Page thumbnail、Note 内 Resource snapshot 、Resource library への新規または更新 Resource を保存する。 |
| 破棄 | 確認後、現在の編集状態を破棄して新しい空 Note を作成する。保存済み Note の実体は削除しない。 |
| ノートファイルをダウンロード | 最新状態を保存した上で portable bundle を生成し、テキストファイルとしてダウンロードする。 |
| ノートファイルをアップロード | portable bundle を読み込み、`import-note` で Resource 実体を復元して保存する。 |
| Publish | 公開用 Note を保存し、公開 URL を返す。公開 URL で読み込まれた Note は閲覧専用モードになる。 |
| ログアウト | ログイン状態を解除する。ログインが必要な保存、公開、アップロード操作は利用できなくなる。 |

## 20.2. Page メニュー

| 操作 | データ更新 |
|---|---|
| 一覧 | Page 一覧画面を表示する。コマンドリストメニューは閉じる。 |
| 名前 | 現在編集中の Page の名称を定義または変更する。 |
| コピー | 現在編集中の Page と同じ内容を持つ Page を、最終 Page として追加する。コピーされた Page には新しい Page ID を割り当てる。 |
| ページ作成 | 白紙の Page を最終 Page として追加する。新しい Page ID を割り当てる。 |

Page 番号は `pages[]` の表示順序から決まる。 Page ID は `_page_uuid` であり、Page の追加、削除、並べ替えによって Page 番号が変わっても同じ Page を識別する。

## 20.3. New メニュー

New メニューは、現在の Page に新しい Content、Topic、Memo を追加する。 追加された Node は、本書で定義する新モデル属性のみを持つ。 旧モデル属性を併せ持つ互換データとして生成してはならない。

## 20.4. Group メニュー

Group メニューは、選択中 Node の整列、Group 定義、Group 解除、コピー、編集を行う。 Group の表示、非表示、bloom、wilt は Group 全体を単位として扱う。 Timeline Group も同じ一括表示ルールに従う。 == Portable Note bundle と Resource 実体の扱い

通常保存では、upload 済み original ファイルを Note snapshot に複製しない。`role=original` は次の upload 領域を参照する。

```
data/{user_uuid}/upload/YYYY/MM/DD/{uploaded_file_uuid}/{filename}
```

Note snapshot には、画面表示の維持に必要な `resource.json`、`thumbnail.`、`preview.pdf` を保存する。標準では `original.` は作成しない。

```
data/{user_uuid}/note/YYYY/MM/DD/{note_uuid}/resource/{resource_uuid}/resource.json
data/{user_uuid}/note/YYYY/MM/DD/{note_uuid}/resource/{resource_uuid}/thumbnail.*
data/{user_uuid}/note/YYYY/MM/DD/{note_uuid}/resource/{resource_uuid}/preview.pdf
```

ノートをテキストファイルとしてダウンロードするときは、持ち運びに必要な original / thumbnail / preview を `bundle.files[]` に Base64 で同梱する。bundle の `path` はユーザーUUIDを含まない論理パスとする。

```
upload/YYYY/MM/DD/{uploaded_file_uuid}/{filename}
note/resource/{resource_uuid}/thumbnail.*
note/resource/{resource_uuid}/preview.pdf
```

インポート時は `upload/…` を `data/{current_user_uuid}/upload/…` に、`note/resource/…` を新しい Note snapshot 配下に展開し、`resource.storage.files[].sourcePath` を復元後の実パスに更新する。

# Chapter 21. 2026-04-29 補足: ノート入出力 API とメニューコマンド

ノート入出力では、通常保存、保存済みノートの読み込み、ノートファイルのダウンロード、ノートファイルのアップロード/復元を分ける。 portable bundle の復元保存は、通常保存用の `save-note` ではなく、専用入口 `import-note` で受け付ける。

| メニュー/操作 | **JavaScript** | ローカル **Python** | **Linux CGI** |
|---|---|---|---|
| ノート > 保存 | `wuwei.note.saveNote()` | `cgi-bin/save-note.py` | `server/save-note.cgi` |
| ノート > 開く | `wuwei.note.loadNote()` | `cgi-bin/load-note.py` | `server/load-note.cgi` |
| ノート > ノートファイルをダウンロード | `wuwei.menu.note.downloadFile()` → `saveNote()` → `exportPortableNoteText()` | `save-note.py` 後、`load-note.py` に `bundle=1` | `save-note.cgi` 後、`load-note.cgi` に `bundle=1` |
| ノート > ノートファイルをアップロード | `wuwei.menu.note.importFile()` → `wuwei.note.importNote()` | `cgi-bin/import-note.py` | `server/import-note.cgi` |
| ノート > Publish | `wuwei.menu.publish.publish()` → `saveNote()` → `publishNote()` | `save-note.py` 後、`publish-note.py` | `save-note.cgi` 後、`publish-note.cgi` |

`load-note` は保存済み `note.json` を読み込む API である。 ノートファイルのダウンロードでは、`load-note` に `bundle=1` を指定し、アップロード original、thumbnail、preview を `bundle.files[]` に Base64 で同梱した portable note text を取得する。

`import-note` は、利用者がアップロードした portable note text の `bundle.files[]` を復元する API である。 `upload/…` は `data/{current_user_uuid}/upload/…` に展開し、`note/resource/…` は新しい Note snapshot 配下に展開する。 復元後の Note JSON からは `bundle`、`portable`、`resourceBundle` を削除して保存する。